

# Can your treasury pay the team?

Runway Guard turns crypto balances into dated operating coverage.

## The problem is a payment deadline

A token balance can look healthy while payroll depends on falling prices, a depegged stablecoin or locked assets. Small crypto-native teams need to connect market value to the dollars they owe, on the dates they owe them.

## A focused workflow, built by Shivam Gupta

Enter balances or import CSV. Schedule recurring payroll inside monthly burn and add one-off costs. Refresh CoinMarketCap quotes, apply a stress scenario and inspect current versus stressed coverage over 90 days. Sign in, save a private workspace and preserve an exportable decision record.

**\$25,000**

Fiat cash

**\$146,000**

3-month reserve target

**\$121,000**

Fiat reserve gap

**Fictional Northstar Labs sample:** \$42,000 monthly burn includes \$30,000 payroll due in seven days. A \$20,000 infrastructure renewal is due in fourteen days. Its three-month reserve is \$126,000 of burn plus the \$20,000 one-off. Market totals change; these policy inputs do not.

## Why the result is worth trusting

Recurring rows replace their portion of burn; they are not added twice. Locked and scenario-frozen assets cannot fund a bill. Missing matching quotes block readiness; stale or future quotes warn and cannot mark it ready. Cash sums use cents. Payroll dates create visible cliffs rather than a smooth, reassuring average.

The reserve what-if estimates volatile asset units using stressed marks and a fee assumption. It does not trade. Aggregate price and volume cannot guarantee a sale price, depth, settlement or access. The report preserves inputs, quotes, scenario and assumptions for later review.

Try the workflow: [Public GitHub repository](#)

[Live demo - deployment confirmed](#)

Live-data status: real keyless CMC quotes are integrated. Participant campaign-key evidence and registration remain outstanding.

# A lean operating product

A commercial hypothesis with explicit boundaries and a testable path.

## Customer, alternative and differentiation

The first customer hypothesis is a founder or fractional finance lead at a 5-50-person crypto team. The recurring job is a payment-cycle readiness check and a stakeholder report. Wallets, payment platforms, managed treasury providers and spreadsheets are real alternatives. Runway Guard positions itself as a narrow planning companion before execution; it does not claim to have invented treasury risk analysis.

## Proposed \$29/month Operator plan

This price is a hypothesis, not an available paid subscription. Personal workspaces, saved reports and exports are implemented. Billing, scheduled checks and collaboration are future work. Validate demand through five observed finance workflows, report-versus-spreadsheet comparisons and paid pilot acceptance. No customer traction, revenue or willingness to pay is claimed.

## Low marginal compute; licensing before launch

The core engine is deterministic and uses no LLM tokens. A shared five-minute quote cache would budget  $12 \times 24 \times 30 = 8,640$  calls per month. A five-asset, single-USD batch fits the documented one-credit-per-250-assets quote rule. Retries, hosting, storage, backups and support need separate budgets. This is a design estimate, not a deployed background scheduler. [CMC quote reference](#)

Confirm CMC licensing for paid B2B use before commercial launch. The pricing FAQ permits integration but describes personal, non-commercial end-user use; that ambiguity needs resolution. Free tiers can support a pilot, but do not prove a sustainable production cost. [CMC pricing and licensing FAQ](#)

## What the CMC integration adds

**GET /v3/cryptocurrency/quotes/latest** batches stable CMC IDs for BTC, ETH, SOL, USDC and USDT in USD. The server parses V3 asset/quote arrays and exposes response evidence and timestamps. Current keyless access is real CMC data; authenticated participant-key evidence is a separate submission requirement. [Keyless public API reference](#)

## Evidence package and remaining submission work

The live deployment is confirmed. Code includes quote inputs, scenarios, reserve preview, sign-in, private workspace/report history and JSON/printable HTML exports. To complete the submission: verify saved-account workflows; obtain campaign-key call evidence and registration; record and upload the narrated demo; submit on DoraHacks; post the submission and video on X with #BuildwithCMC. [Official hackathon requirements](#)

Deadline: 30 September 2026, 23:59 UTC = 1 October, 05:29 IST. Preserve honest evidence: account workflows need end-to-end verification; no execution, traction or completed registration should be implied without proof.